

AlgorithmPart-02

Breadth First Search or BFS for a Graph

Breadth First Search (BFS) is a fundamental **graph traversal algorithm**. It involves visiting all the connected nodes of a graph in a level-by-level manner.

Breadth First Search (BFS) for a Graph:

Breadth First Search (BFS) is a graph traversal algorithm that explores all the vertices in a graph at the current depth before moving on to the vertices at the next depth level. It starts at a specified vertex and visits all its neighbors before moving on to the next level of neighbors. **BFS** is commonly used in algorithms for pathfinding, connected components, and shortest path problems in graphs.

Breadth First Search (BFS) for a Graph Algorithm:

Let's discuss the algorithm for the BFS

1.Initialization: Add the starting node to the queue and mark it as visited.

2.Exploration: While the queue is not empty:

- Remove a node from the queue (dequeue) and visit it.
- For each unvisited neighbor of the removed node:
- Add the neighbor to the queue (enqueue).
- Mark the neighbor as visited.

3.Termination: Repeat step 2 until the queue is empty.

The BFS algorithm explores the graph level by level, visiting all the nodes at the current level before moving on to the next level. This ensures that the shortest path from the starting node to each reachable node is found.

Relation between BFS for Graph and BFS for Tree:

Breadth-First Traversal (BFS) for a graph is similar to the Breadth-First Traversal of a tree.

The only catch here is, that, unlike **trees**, **graphs** may contain cycles, so we may come to the same node again. To avoid processing a node more than once, we divide the vertices into two categories:

- Visited and
- Not visited.

A boolean visited array is used to mark the visited vertices. For simplicity, it is assumed that all vertices are reachable from the starting vertex. BFS **uses** a **queue data structure** for traversal.

How Does the BFS Algorithm Work?

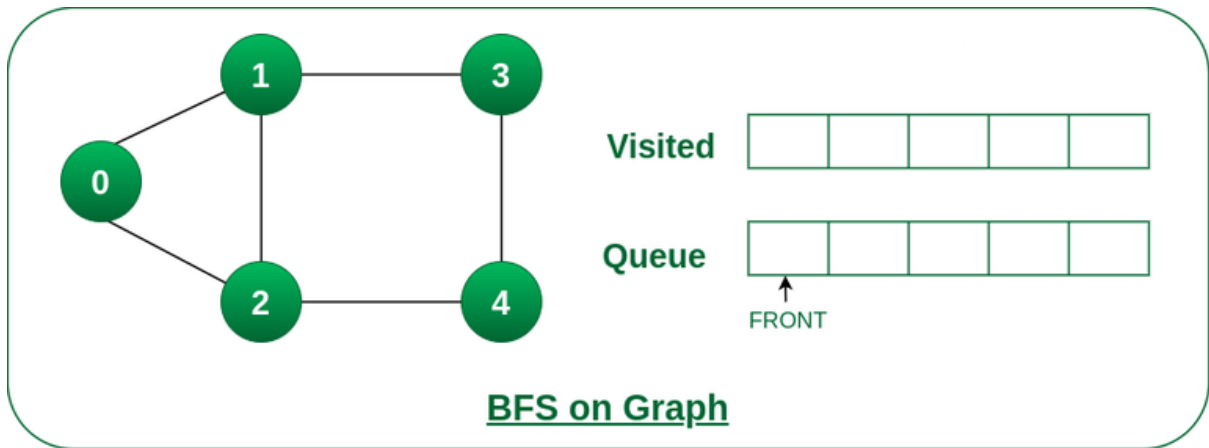
Starting from the root, all the nodes at a particular level are visited first and then the nodes of the next level are traversed till all the nodes are visited.

To do this a queue is used. All the adjacent unvisited nodes of the current level are pushed into the queue and the nodes of the current level are marked visited and popped from the queue.

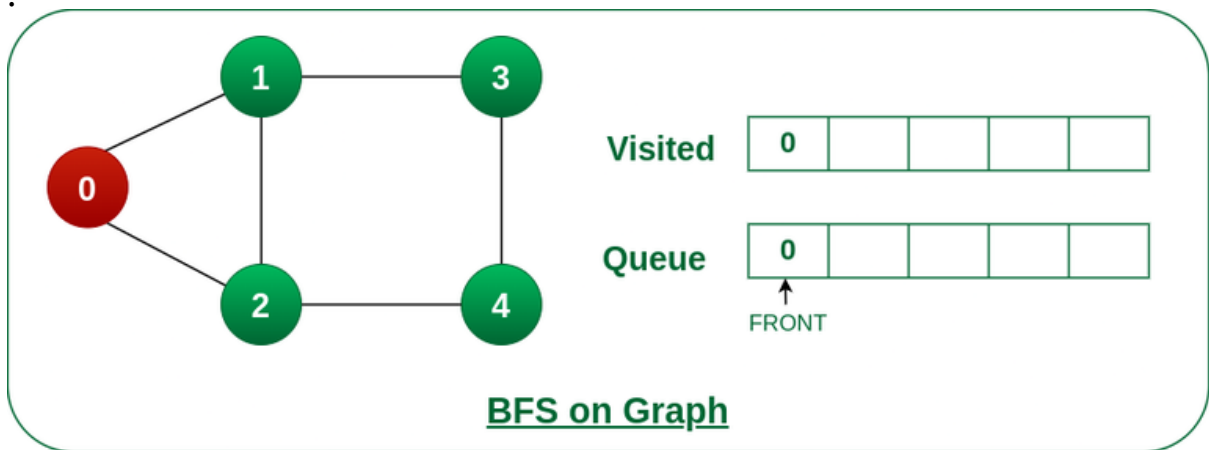
Illustration:

Let us understand the working of the algorithm with the help of the following example.

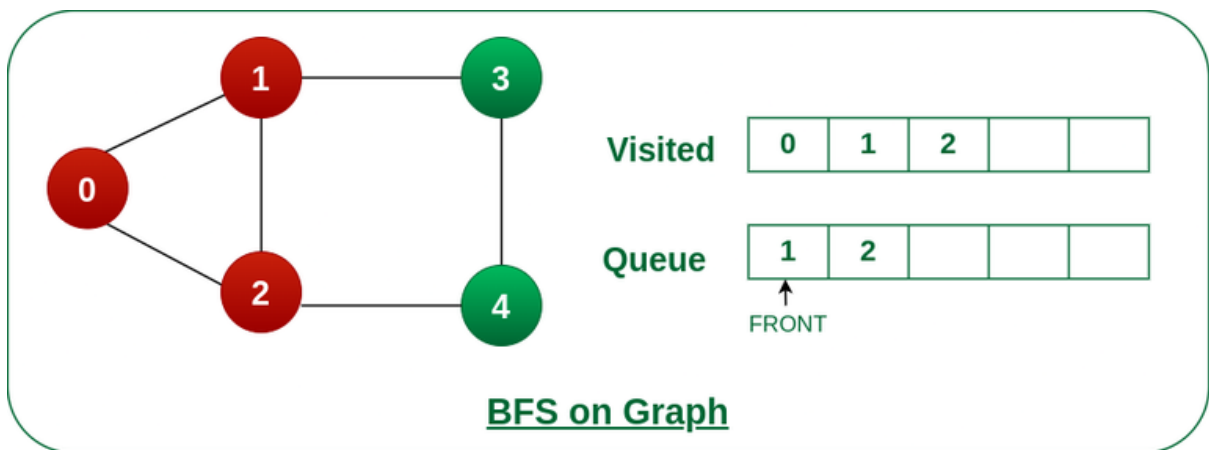
Step1: Initially queue and visited arrays are empty.



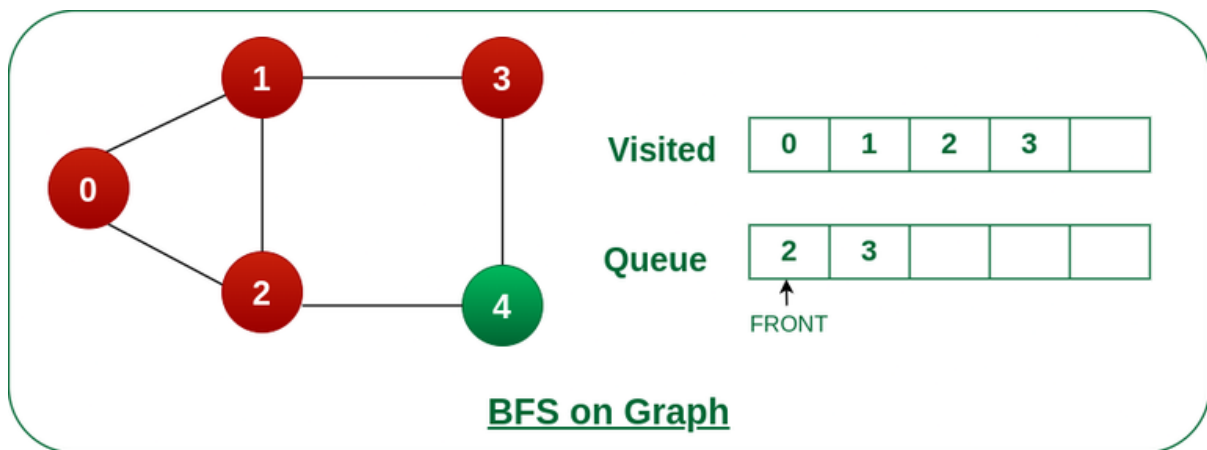
Step2: Push node 0 into queue and mark it visited



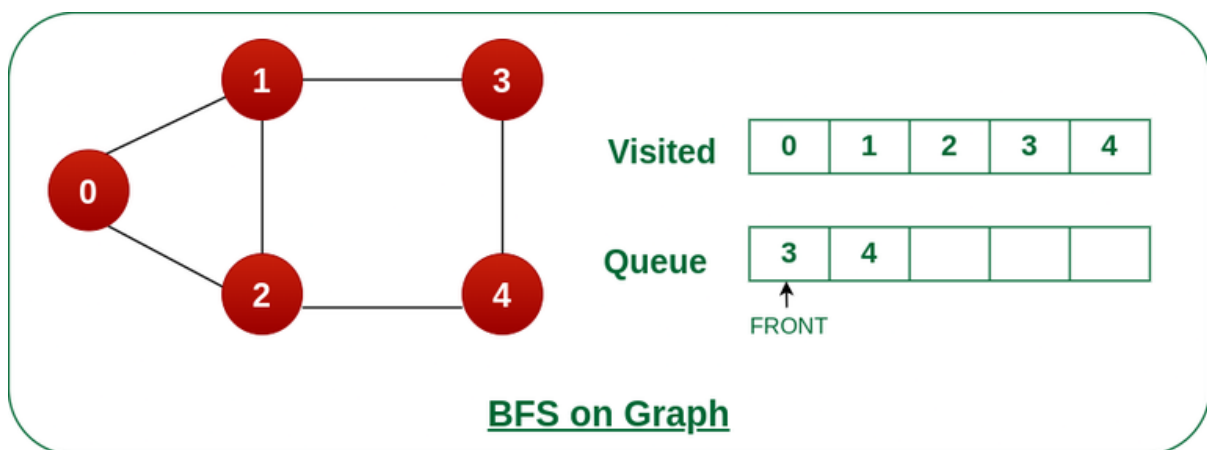
Step 3: Remove node 0 from the front of queue and visit the unvisited neighbours and push them into queue.



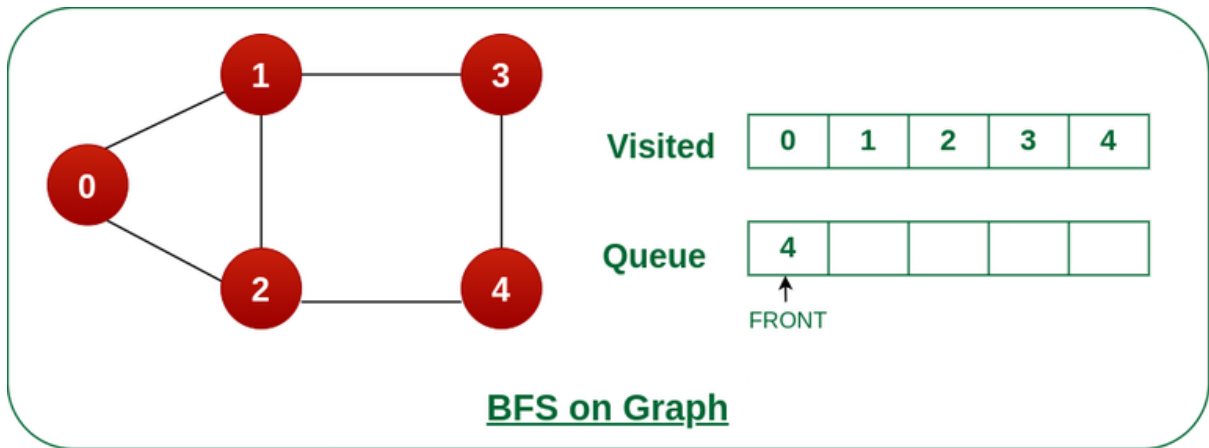
Step 4: Remove node 1 from the front of queue and visit the unvisited neighbours and push them into queue.



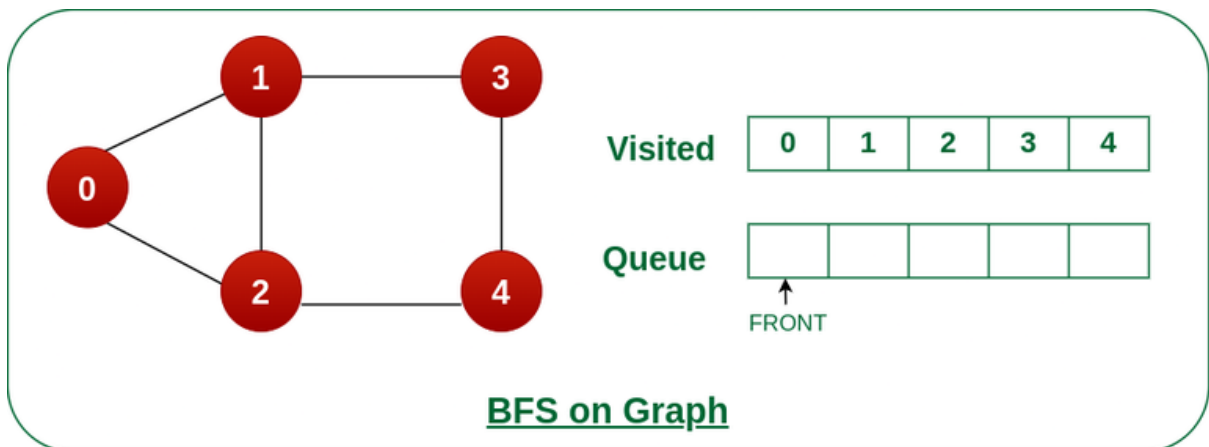
Step 5: Remove node 2 from the front of queue and visit the unvisited neighbours and push them into queue.



Step 6: Remove node 3 from the front of queue and visit the unvisited neighbours and push them into queue. As we can see that every neighbours of node 3 is visited, so move to the next node that are in the front of the queue.



Steps 7: Remove node 4 from the front of queue and visit the unvisited neighbours and push them into queue.
As we can see that every neighbours of node 4 are visited, so move to the next node that is in the front of the queue.



Now, Queue becomes empty, So, terminate these process of iteration.

Pseudocode:

BFS(graph, startNode):

create an empty queue queue

create a set of visited nodes visited

add startNode to queue

add startNode to visited

while queue is not empty:

currentNode = dequeue from queue

for each unvisited neighbor of currentNode in graph:

add neighbor to queue

add neighbor to visited

output or process currentNode

Time Complexity: $O(V+E)$, where V is the number of nodes and E is the number of edges.

Auxiliary Space: $O(V)$

Complexity Analysis of Breadth-First Search (BFS) Algorithm:

Time Complexity of BFS Algorithm: $O(V + E)$

- BFS explores all the vertices and edges in the graph. In the worst case, it visits every vertex and edge once. Therefore, the time complexity of BFS is $O(V + E)$, where V and E are the number of vertices and edges in the given graph.

Space Complexity of BFS Algorithm: $O(V)$

- BFS uses a queue to keep track of the vertices that need to be visited. In the worst case, the queue can contain all the vertices in the graph. Therefore, the space complexity of BFS is $O(V)$, where V and E are the number of vertices and edges in the given graph.

Applications of BFS in Graphs:

BFS has various applications in graph theory and computer science, including:

- **Shortest Path Finding:** BFS can be used to find the shortest path between two nodes in an unweighted graph. By keeping track of the parent of each node during the traversal, the shortest path can be reconstructed.
- **Cycle Detection:** BFS can be used to detect cycles in a graph. If a node is visited twice during the traversal, it indicates the presence of a cycle.
- **Connected Components:** BFS can be used to identify connected components in a graph. Each connected component is a set of nodes that can be reached from each other.
- **Topological Sorting:** BFS can be used to perform topological sorting on a directed acyclic graph (DAG). Topological sorting arranges the nodes in a linear order such that for any edge (u, v) , u appears before v in the order.
- **Level Order Traversal of Binary Trees:** BFS can be used to perform a level order traversal of a binary tree. This traversal visits all nodes at the same level before moving to the next level.

- **Network Routing:** BFS can be used to find the shortest path between two nodes in a network, making it useful for routing data packets in network protocols.

Question 1: What is BFS and how does it work?

Answer: BFS is a graph traversal algorithm that systematically explores a graph by visiting all the vertices at a given level before moving on to the next level. It starts from a starting vertex, enqueues it into a queue, and marks it as visited. Then, it dequeues a vertex from the queue, visits it, and enqueues all its unvisited neighbors into the queue. This process continues until the queue is empty.

Question 2: What are the applications of BFS?

Answer: BFS has various applications, including finding the shortest path in an unweighted graph, detecting cycles in a graph, topologically sorting a directed acyclic graph (DAG), finding connected components in a graph, and solving puzzles like mazes and Sudoku.

Question 3: What is the time complexity of BFS?

Answer: The time complexity of BFS is $O(V + E)$, where V is the number of vertices and E is the number of edges in the graph.

Question 4: What is the space complexity of BFS?

Answer: The space complexity of BFS is $O(V)$, as it uses a queue to keep track of the vertices that need to be visited.

Question 5: What are the advantages of using BFS?

Answer: BFS is simple to implement and efficient for finding the shortest path in an unweighted graph. It also guarantees that all the vertices in the graph are visited.

Depth First Search or DFS for a Graph

Depth First Traversal (or DFS) for a graph is similar to Depth First Traversal of a tree. The only catch here is, that, unlike trees, graphs may contain cycles (a node may be visited twice). To avoid processing a node more than once, use a boolean visited array. A graph can have more than one DFS traversal.

How does DFS work?

Depth-first search is an algorithm for traversing or searching tree or graph data structures. The algorithm starts at the root node (selecting some arbitrary node as the root node in the case of a graph) and explores as far as possible along each branch before backtracking.

Depth First Search (DFS) for a Graph Algorithm:

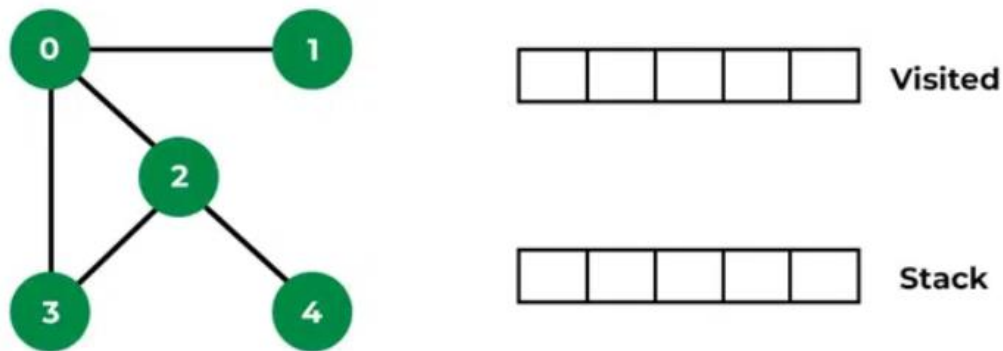
The steps of the DFS algorithm are:

1. Initialize an empty stack and a set to store visited nodes.
2. Push the starting node onto the stack and add it to the visited set.
3. While the stack is not empty:
 - Pop a node from the stack.
 - Process the node (e.g., print it, perform some operation on it).
 - For each unvisited neighbor of the popped node: push the neighbor onto the stack.
 - Add the neighbor to the visited set.
4. The algorithm terminates when the stack is empty, meaning all reachable nodes have been visited.

The DFS algorithm explores the graph by going as deep as possible along each branch before backtracking. It uses a stack to keep track of the nodes to visit, and the visited set ensures that each node is visited only once.

Let us understand the working of **Depth First Search** with the help of the following illustration:

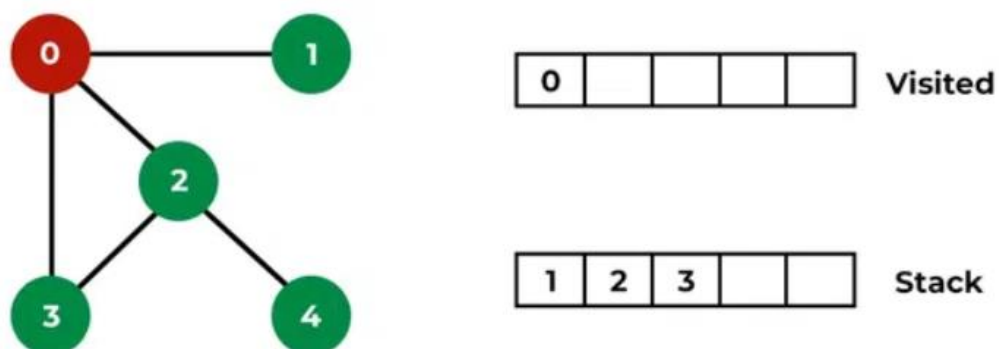
Step1: Initially stack and visited arrays are empty.



DFS on Graph

Stack and visited arrays are empty initially.

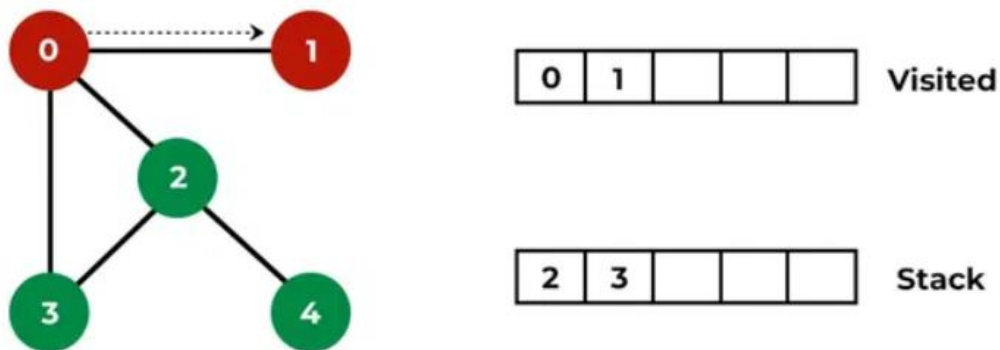
Step 2: Visit 0 and put its adjacent nodes which are not visited yet into the stack.



DFS on Graph

Visit node 0 and put its adjacent nodes (1, 2, 3) into the stack

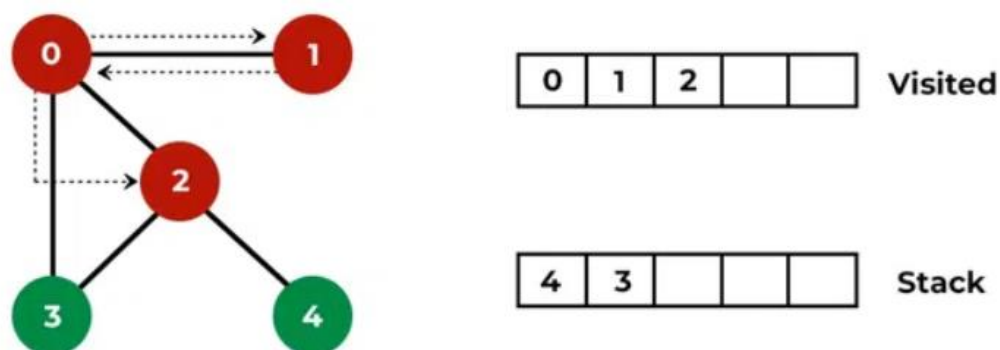
Step 3: Now, Node 1 at the top of the stack, so visit node 1 and pop it from the stack and put all of its adjacent nodes which are not visited in the stack.



DFS on Graph

Visit node 1

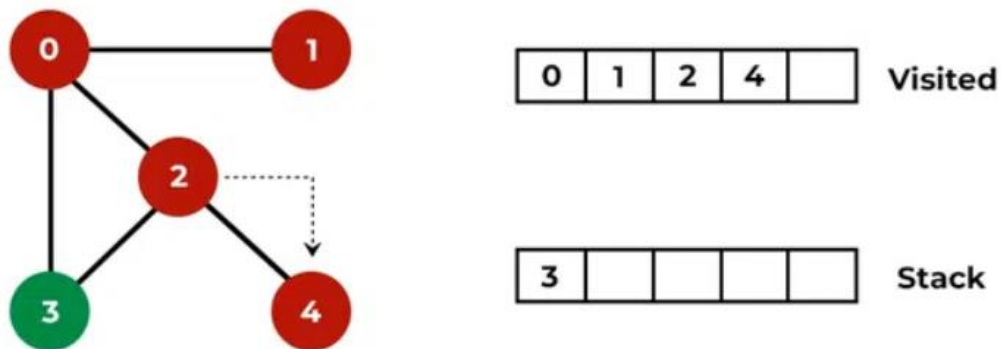
Step 4: Now, Node 2 at the top of the stack, so visit node 2 and pop it from the stack and put all of its adjacent nodes which are not visited (i.e, 3, 4) in the stack.



DFS on Graph

Visit node 2 and put its unvisited adjacent nodes (3, 4) into the stack

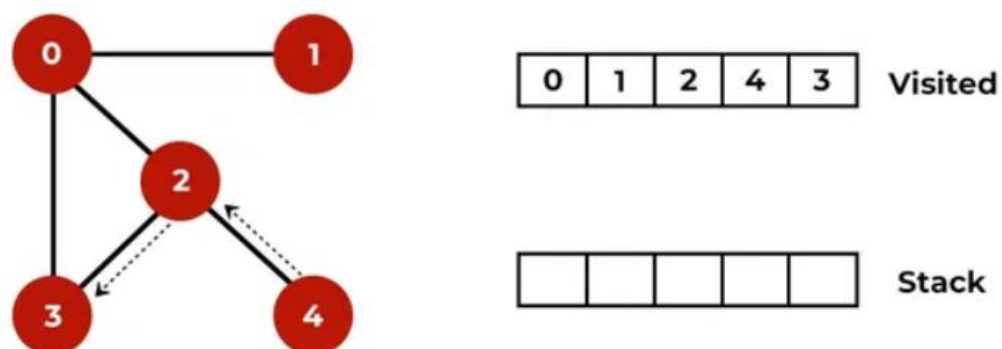
Step 5: Now, Node 4 at the top of the stack, so visit node 4 and pop it from the stack and put all of its adjacent nodes which are not visited in the stack.



DFS on Graph

Visit node 4

Step 6: Now, Node 3 at the top of the stack, so visit node 3 and pop it from the stack and put all of its adjacent nodes which are not visited in the stack.



DFS on Graph

Visit node 3

Now, Stack becomes empty, which means we have visited all the nodes and our DFS traversal ends.

Pseudocode:

DFS(graph, startNode):

 Initialize an empty set or list 'visited'

 Call DFS_Visit(startNode, visited)

DFS_Visit(node, visited):

 Mark 'node' as visited and print 'node'

 For each neighbor of 'node' in 'graph':

 If neighbor is not in 'visited':

 Call DFS_Visit(neighbor, visited)

Complexity Analysis of Depth First Search:

- **Time complexity:** $O(V + E)$, where V is the number of vertices and E is the number of edges in the graph.
- **Auxiliary Space:** $O(V + E)$, since an extra visited array of size V is required, And stack size for iterative call to DFS function.

Comparison of Dijkstra's algorithm, Bellman-Ford algorithm, and Floyd-Warshall algorithm:

Dijkstra's Algorithm:

1. Used for finding the shortest path between a single source node and all other nodes in a weighted graph.
2. Assumes that all edge weights are non-negative.
3. Time complexity: $O((V+E)\log V)$ using a priority queue, where V is the number of vertices and E is the number of edges.
4. Works by repeatedly selecting the vertex with the smallest known distance from the source and updating the distances of its neighbors.

Bellman-Ford Algorithm:

1. Used for finding the shortest path between a single source node and all other nodes in a weighted graph, even when the graph contains negative-weight edges.
2. Time complexity: $O(V * E)$, where V is the number of vertices and E is the number of edges.
3. Works by iteratively relaxing all edges, updating the distance of each node from the source.
4. Can detect negative-weight cycles in the graph and report if a path doesn't exist.

Floyd-Warshall Algorithm:

1. Used for finding the shortest paths between all pairs of nodes in a weighted graph.
2. Handles both positive and negative edge weights, but cannot handle negative-weight cycles.
3. Time complexity: $O(V^3)$, where V is the number of vertices.
4. Works by iteratively updating the distance matrix, using each node as a potential intermediate node in the shortest path.
5. Computes the all-pairs shortest paths in a single pass.

Key differences:

- ❖ Dijkstra's algorithm is more efficient than Bellman-Ford for graphs without negative-weight edges, but Bellman-Ford can handle negative-weight edges.
- ❖ Bellman-Ford can detect negative-weight cycles, while Dijkstra's algorithm and Floyd-Warshall cannot.
- ❖ Floyd-Warshall computes the all-pairs shortest paths, while Dijkstra's and Bellman-Ford algorithms find the shortest paths from a single source to all other nodes.
- ❖ Floyd-Warshall has a higher time complexity of $O(V^3)$, compared to Dijkstra's $O((V+E)\log V)$ and Bellman-Ford's $O(V * E)$.
- ❖ The choice of algorithm depends on the specific requirements of the problem, such as the presence of negative-weight edges, the need for all-pairs shortest paths, and the size of the graph.

Greedy Algorithms

What is Greedy Algorithm?

A **greedy algorithm** is a type of optimization algorithm that makes locally optimal choices at each step to find a globally optimal solution. It operates on the principle of “taking the best option now” without considering the long-term consequences.

Steps for Creating a Greedy Algorithm

The steps to define a greedy algorithm are:

1. **Define the problem:** Clearly state the problem to be solved and the objective to be optimized.
2. **Identify the greedy choice:** Determine the locally optimal choice at each step based on the current state.
3. **Make the greedy choice:** Select the greedy choice and update the current state.
4. **Repeat:** Continue making greedy choices until a solution is reached.

Following the given steps, one can learn how to use greedy algorithms to find optimal solutions.

Greedy Algorithm Examples

Examples of greedy algorithms are the best way to understand the algorithm. Some greedy algorithm real-life examples are:

- **Fractional Knapsack:** Optimizes the value of items that can be fractionally included in a knapsack with limited capacity.
- **Dijkstra's algorithm:** Finds the shortest path from a source vertex to all other vertices in a weighted graph.

- [Kruskal's algorithm](#): Finds the minimum spanning tree of a weighted graph.
- [Huffman coding](#): Compresses data by assigning shorter codes to more frequent symbols.

Applications of Greedy Algorithm

There are many **applications of the greedy method in DAA**. Some important greedy algorithm applications are:

- Assigning tasks to resources to minimize waiting time or maximize efficiency.
- Selecting the most valuable items to fit into a knapsack with limited capacity.
- Dividing an image into regions with similar characteristics.
- Reducing the size of data by removing redundant information.

Disadvantages/Limitations of Using a Greedy Algorithm

Below are some disadvantages of the Greedy Algorithm:

- Greedy algorithms may not always find the best possible solution.
- The order in which the elements are considered can significantly impact the outcome.
- Greedy algorithms focus on local optimizations and may miss better solutions that require considering a broader context.
- Greedy algorithms are not applicable to problems where the greedy choice does not lead to an optimal solution.

Dynamic Programming or DP

What is Dynamic Programming (DP)?

Dynamic Programming (DP) is a method used in mathematics and computer science to solve complex problems by breaking them down into simpler subproblems. By solving each subproblem only once and storing the results, it avoids redundant computations, leading to more efficient solutions for a wide range of problems.

How Does Dynamic Programming (DP) Work?

- **Identify Subproblems:** Divide the main problem into smaller, independent subproblems.
- **Store Solutions:** Solve each subproblem and store the solution in a table or array.
- **Build Up Solutions:** Use the stored solutions to build up the solution to the main problem.
- **Avoid Redundancy:** By storing solutions, DP ensures that each subproblem is solved only once, reducing computation time.

Examples of Dynamic Programming (DP)

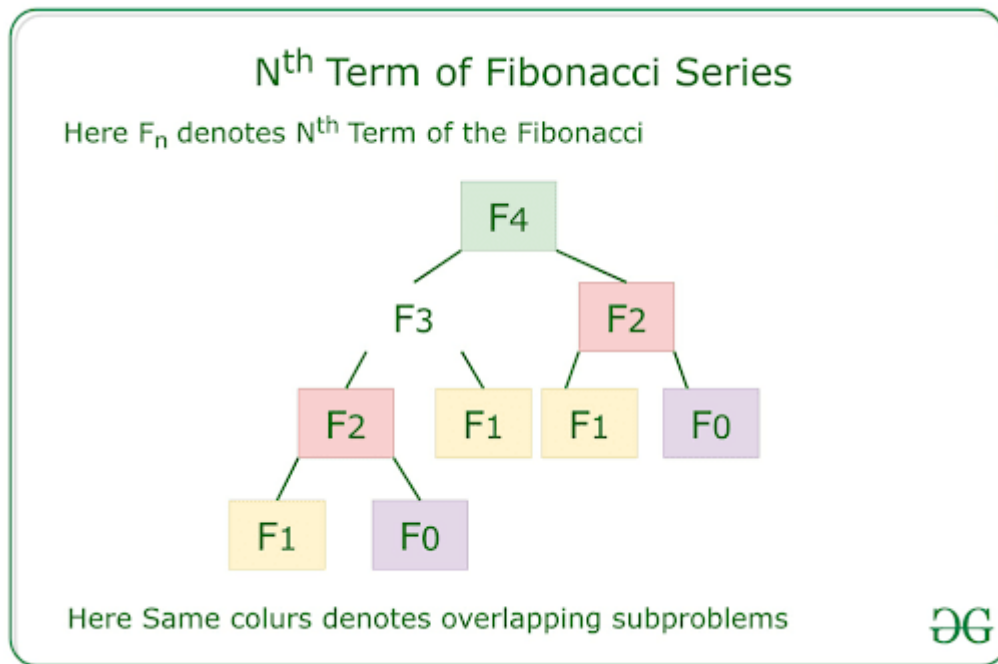
Example 1: Consider the problem of finding the Fibonacci sequence:

Fibonacci sequence: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, ...

Brute Force Approach:

To find the n th Fibonacci number using a brute force approach, you would simply add the $(n-1)$ th and $(n-2)$ th Fibonacci numbers. This would work, but it would be inefficient for large values of n , as it would require calculating all the previous Fibonacci numbers.

Dynamic Programming Approach:



Fibonacci Series using Dynamic Programming

- **Subproblems:** $F(0)$, $F(1)$, $F(2)$, $F(3)$, ...
- **Store Solutions:** Create a table to store the values of $F(n)$ as they are calculated.
- **Build Up Solutions:** For $F(n)$, look up $F(n-1)$ and $F(n-2)$ in the table and add them.
- **Avoid Redundancy:** The table ensures that each subproblem (e.g., $F(2)$) is solved only once.

By using DP, we can efficiently calculate the Fibonacci sequence without having to recompute subproblems.

Example 2: Longest common subsequence (finding the longest subsequence that is common to two strings)

Example 3: Shortest path in a graph (finding the shortest path between two nodes in a graph)

Example 4: Knapsack problem (finding the maximum value of items that can be placed in a knapsack with a given capacity)

Why is Memoization used?

Memoization is a specific form of *caching* that is used in *dynamic programming*. The purpose of caching is to improve the performance of our programs and keep data accessible that can be used later. It basically stores the previously calculated result of the subproblem and uses the stored result for the same subproblem. This removes the extra effort to calculate again and again for the same problem. And we already know that if the same problem occurs again and again, then that problem is recursive in nature.

Where to use Memoization?

We can use the memoization technique where the use of the previously-calculated results comes into the picture. This kind of problem is mostly used in the context of recursion, especially with problems that involve overlapping subproblems.

Example to show where to use memoization:

Let us try to find the factorial of a number.

Below is a **recursive method** for finding the factorial of a number:

```
int factorial(unsigned int n)
{
    if (n == 0)
        return 1;
    return n * factorial(n - 1);
}
```

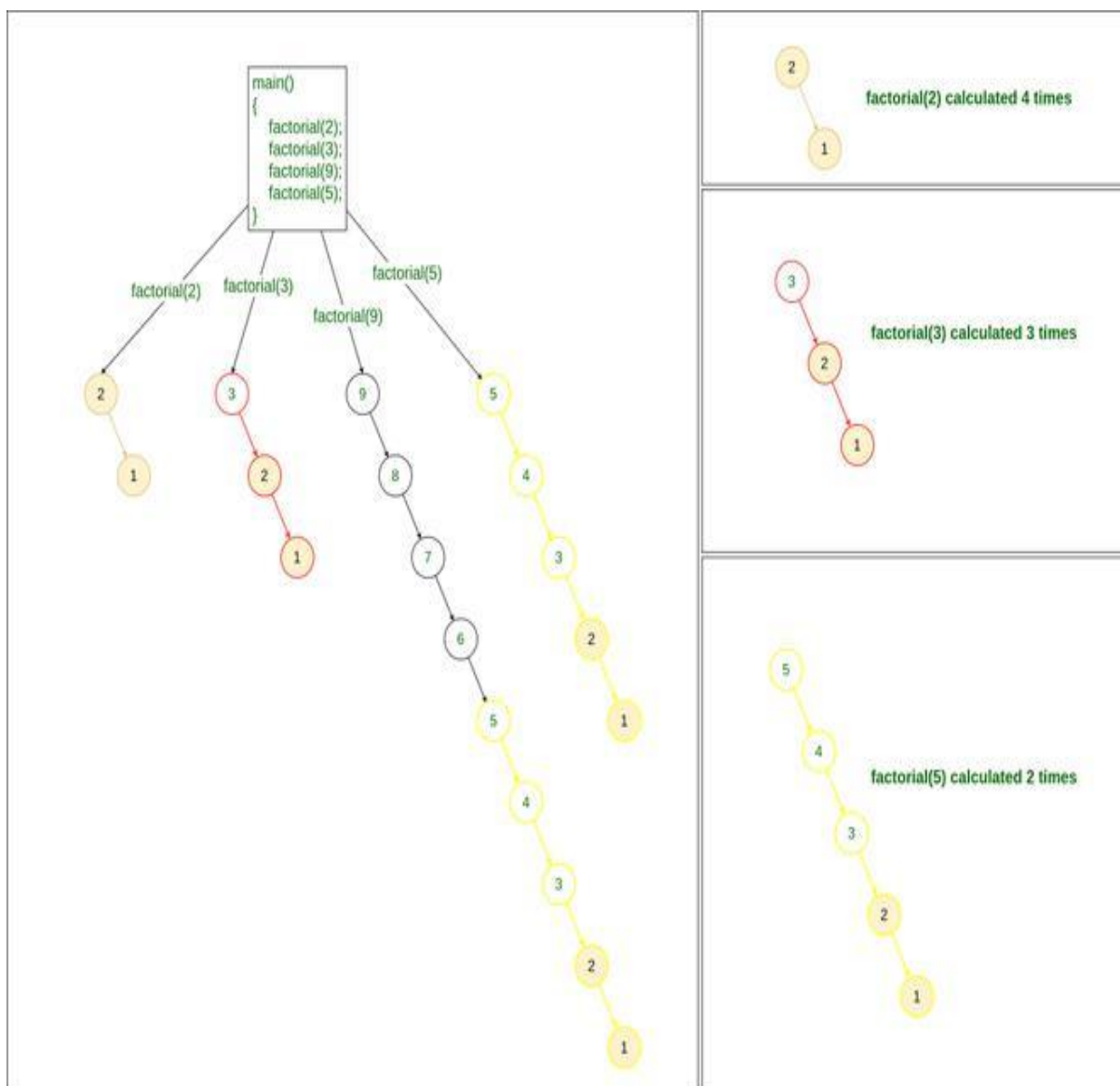
What happens if we use this recursive method?

If you write the complete code for the above snippet, you will notice that there will be 2 methods in the code:

```
1. factorial(n)
2. main()
```

Now if we have multiple queries to find the factorial, such as finding factorial of 2, 3, 9, and 5, then we will need to call the factorial() method 4 times:

```
factorial(2)
factorial(3)
factorial(9)
factorial(5)
```



Recursive method to find Factorial

So it is safe to say that for finding factorial of numbers K numbers, the time complexity needed will be $O(N*K)$

- $O(N)$ to find the factorial of a particular number, and
- $O(K)$ to call the factorial() method K different times.

How Memoization can help with such problems?

If we notice in the above problem, while calculation factorial of 9:

- We are calculating the factorial of 2
- We are also calculating the factorial of 3,
- and We are calculating the factorial of 5 as well

Therefore if we store the result of each individual factorial at the first time of calculation, we can easily return the factorial of any required number in just $O(1)$ time. This process is known as **Memoization**.

Solution using Memoization (How does memoization work?):

If we find the factorial of 9 first and store the results of individual sub-problems, we can easily print the factorial of each input in $O(1)$.

Therefore the time complexity to find factorial numbers using memoization will be $O(N)$

- $O(N)$ to find the factorial of the largest input
- $O(1)$ to print the factorial of each input.

Pseudocode:

```
function factorial(n):  
    if n == 0 or n == 1:  
        return 1  
    else:  
        if n has been computed before:  
            return the previously computed value  
        else:  
            result = n * factorial(n-1)  
            store the computed value for n  
            return result
```

When to Use Dynamic Programming (DP)?

Dynamic programming is an optimization technique used when solving problems that consists of the following characteristics:

1. Optimal Substructure:

Optimal substructure means that we combine the optimal results of subproblems to achieve the optimal result of the bigger problem.

Example:

Consider the problem of finding the **minimum cost** path in a weighted graph from a **source** node to a **destination** node. We can break this problem down into smaller subproblems:

- Find the **minimum cost** path from the **source** node to each **intermediate** node.
- Find the **minimum cost** path from each **intermediate** node to the **destination** node.

The solution to the larger problem (finding the minimum cost path from the source node to the destination node) can be constructed from the solutions to these smaller subproblems.

2. Overlapping Subproblems:

The same subproblems are solved repeatedly in different parts of the problem.

Example:

Consider the problem of computing the **Fibonacci series**. To compute the Fibonacci number at index **n**, we need to compute the Fibonacci numbers at indices **n-1** and **n-2**. This means that the subproblem of computing the Fibonacci number at index **n-1** is used twice in the solution to the larger problem of computing the Fibonacci number at index **n**.

Approaches of Dynamic Programming (DP)

Dynamic programming can be achieved using two approaches:

1. Top-Down Approach (Memoization):

In the top-down approach, also known as **memoization**, we start with the final solution and recursively break it down into smaller subproblems. To avoid redundant calculations, we store the results of solved subproblems in a memoization table.

Let's breakdown Top down approach:

- Starts with the final solution and recursively breaks it down into smaller subproblems.
- Stores the solutions to subproblems in a table to avoid redundant calculations.
- Suitable when the number of subproblems is large and many of them are reused.

2. Bottom-Up Approach (Tabulation):

In the bottom-up approach, also known as **tabulation**, we start with the smallest subproblems and gradually build up to the final solution. We store the results of solved subproblems in a table to avoid redundant calculations.

Let's breakdown Bottom-up approach:

- Starts with the smallest subproblems and gradually builds up to the final solution.
- Fills a table with solutions to subproblems in a bottom-up manner.
- Suitable when the number of subproblems is small and the optimal solution can be directly computed from the solutions to smaller subproblems.

Dynamic Programming (DP) Algorithm

Dynamic programming is a algorithmic technique that solves complex problems by breaking them down into smaller subproblems and storing their solutions for future use. It is particularly effective for problems that contains **overlapping subproblems** and **optimal substructure**.

Common Algorithms that Use Dynamic Programming:

- **Longest Common Subsequence (LCS):** Finds the longest common subsequence between two strings.

- **Shortest Path in a Graph:** Finds the shortest path between two nodes in a graph.
- **Knapsack Problem:** Determines the maximum value of items that can be placed in a knapsack with a given capacity.
- **Matrix Chain Multiplication:** Optimizes the order of matrix multiplication to minimize the number of operations.
- **Fibonacci Sequence:** Calculates the nth Fibonacci number.

Advantages of Dynamic Programming (DP)

Dynamic programming has a wide range of advantages, including:

- Avoids recomputing the same subproblems multiple times, leading to significant time savings.
- Ensures that the optimal solution is found by considering all possible combinations.
- Breaks down complex problems into smaller, more manageable subproblems.

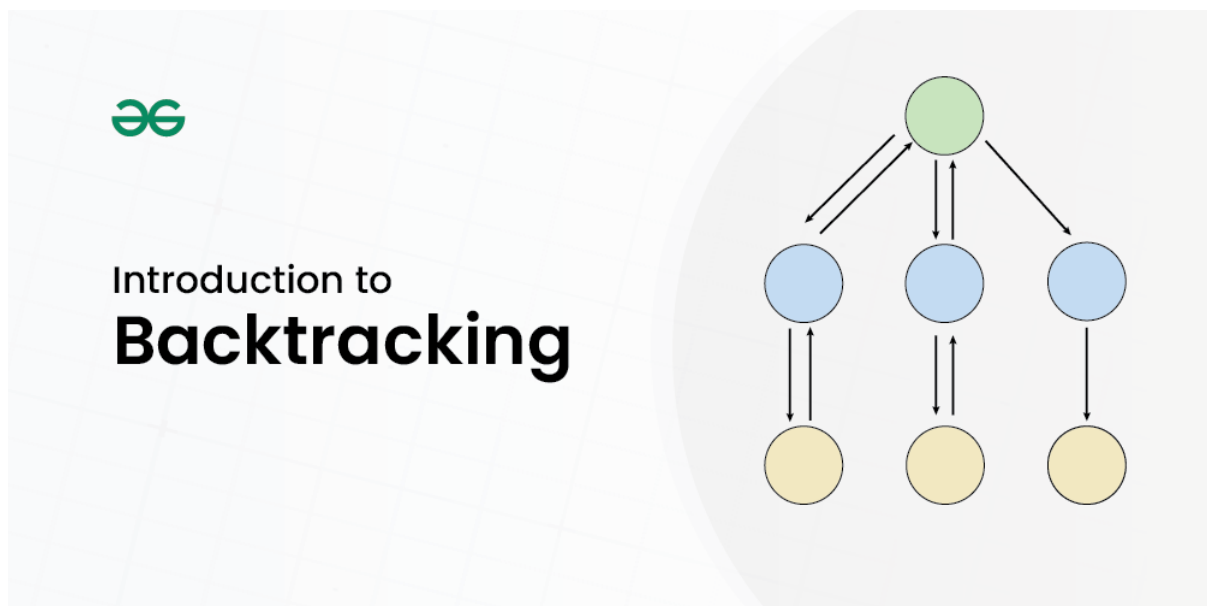
Applications of Dynamic Programming (DP)

Dynamic programming has a wide range of applications, including:

- **Optimization:** Knapsack problem, shortest path problem, maximum subarray problem
- **Computer Science:** Longest common subsequence, edit distance, string matching
- **Operations Research:** Inventory management, scheduling, resource allocation

Backtracking Algorithm

Backtracking algorithms are like problem-solving strategies that help explore different options to find the best solution. They work by trying out different paths and if one doesn't work, they backtrack and try another until they find the right one. It's like solving a puzzle by testing different pieces until they fit together perfectly.



Backtracking

What is Backtracking Algorithm?

Backtracking is a problem-solving algorithmic technique that involves finding a solution incrementally by trying **different options** and **undoing** them if they lead to a **dead end**.

It is commonly used in situations where you need to explore multiple possibilities to solve a problem, like searching for a path in a maze or solving puzzles like **Sudoku**. When a dead end is reached, the algorithm backtracks to the previous decision point and explores a different path until a solution is found or all possibilities have been exhausted.

How Does a Backtracking Algorithm Work?

A **backtracking algorithm** works by recursively exploring all possible solutions to a problem. It starts by choosing an initial solution, and then it explores all possible extensions of that solution. If an extension leads to a solution, the algorithm returns that solution. If an extension does not lead to a solution, the algorithm backtracks to the previous solution and tries a different extension.

The following is a general outline of how a backtracking algorithm works:

1. Choose an initial solution.
2. Explore all possible extensions of the current solution.
3. If an extension leads to a solution, return that solution.
4. If an extension does not lead to a solution, backtrack to the previous solution and try a different extension.
5. Repeat steps 2-4 until all possible solutions have been explored.

Example of Backtracking Algorithm

Example: Finding the shortest path through a maze

Input: A maze represented as a 2D array, where **0** represents an open space and **1** represents a wall.

Algorithm:

1. Start at the starting point.
2. For each of the four possible directions (up, down, left, right), try moving in that direction.
3. If moving in that direction leads to the ending point, return the path taken.
4. If moving in that direction does not lead to the ending point, backtrack to the previous position and try a different direction.

5. Repeat steps 2-4 until the ending point is reached or all possible paths have been explored.

When to Use a Backtracking Algorithm?

Backtracking algorithms are best used to solve problems that have the following characteristics:

- There are multiple possible solutions to the problem.
- The problem can be broken down into smaller subproblems.
- The subproblems can be solved independently.

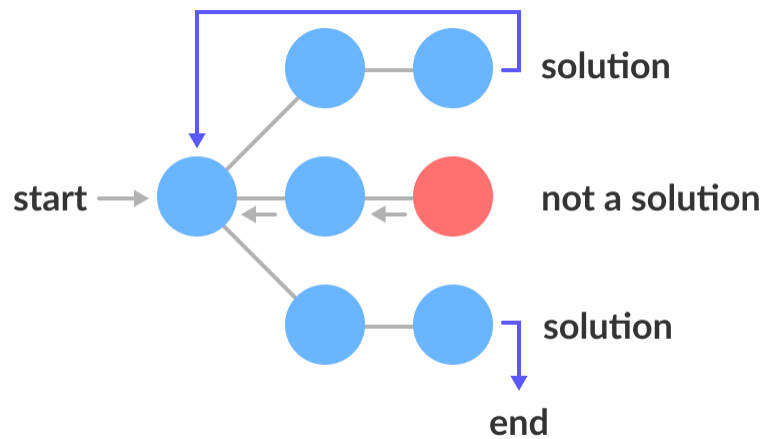
Applications of Backtracking Algorithm

Backtracking algorithms are used in a wide variety of applications, including:

- Solving puzzles (e.g., Sudoku, crossword puzzles)
- Finding the shortest path through a maze
- Scheduling problems
- Resource allocation problems
- Network optimization problems

State Space Tree

A space state tree is a tree representing all the possible states (solution or nonsolution) of the problem from the root as an initial state to the leaf as a terminal state.



State Space Tree

Backtracking Algorithm

```

Backtrack(x)
    if x is not a solution
        return false
    if x is a new solution
        add to list of solutions
    backtrack(expand x)
  
```

The time complexity of Backtracking

The time complexity of backtracking depends on the number of times the function calls itself.

For example, if the function calls itself two times, then its time complexity is $O(2^N)$, and if it calls three times, then $O(3^N)$ and so on. Hence the time complexity of backtracking can be defined as $O(K^N)$, where 'K' is the number of times the function calls itself.

Example Backtracking Approach

Problem: You want to find all the possible ways of arranging 2 boys and 1 girl on 3 benches. Constraint: Girl should not be on the middle bench.

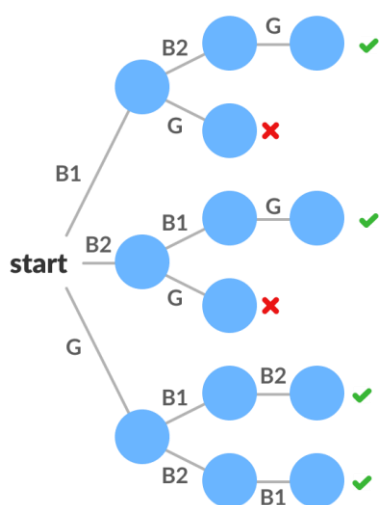
Solution: There are a total of $3! = 6$ possibilities. We will try all the possibilities and get the possible solutions. We recursively try all the possibilities.

All the possibilities are:

B1	B2	G
B1	G	B2
B2	B1	G
B2	G	B1
G	B1	B2
G	B2	B1

All the possibilities

The following **state space tree** shows the possible solutions.



State tree with all the solutions

0/1 Knapsack Problem

Given N items where each item has some weight and profit associated with it and also given a bag with capacity W . The task is to put the items into the bag such that the sum of profits associated with them is the maximum possible.

Note: The constraint here is we can either put an item completely into the bag or cannot put it at all [It is not possible to put a part of an item into the bag].

Examples:

Input: $N = 3$, $W = 4$, $\text{profit[]} = \{1, 2, 3\}$, $\text{weight[]} = \{4, 5, 1\}$

Output: 3

Input: $N = 3$, $W = 3$, $\text{profit[]} = \{1, 2, 3\}$, $\text{weight[]} = \{4, 5, 6\}$

Output: 0

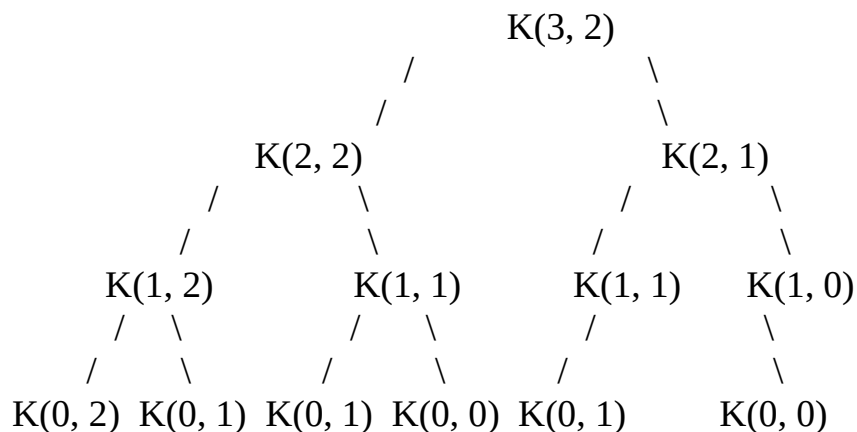
Dynamic Programming Approach for 0/1 Knapsack Problem

Memoization Approach for 0/1 Knapsack Problem:

Note: It should be noted that the above function using recursion computes the same subproblems again and again. See the following recursion tree, $K(1, 1)$ is being evaluated twice.

In the following recursion tree, $\mathbf{K()}$ refers to knapSack() . The two parameters indicated in the following recursion tree are n and W .

The recursion tree is for following sample inputs.
 $\text{weight[]} = \{1, 1, 1\}$, $W = 2$, $\text{profit[]} = \{10, 20, 30\}$



Recursion tree for Knapsack capacity 2 units and 3 items of 1 unit weight.

Bottom-up Approach for 0/1 Knapsack Problem:

To solve the problem follow the below idea:

Since subproblems are evaluated again, this problem has Overlapping Sub-problems property. So the 0/1 Knapsack problem has both properties of a dynamic programming problem. Like other typical Dynamic Programming(DP) problems, re-computation of the same subproblems can be avoided by constructing a temporary array $K[][]$ in a bottom-up manner.

Illustration:

Below is the illustration of the above approach:

Let, $\text{weight[]} = \{1, 2, 3\}$, $\text{profit[]} = \{10, 15, 40\}$, Capacity = 6

- If no element is filled, then the possible profit is 0.

weight→ item↓/	0	1	2	3	4	5	6
0	0	0	0	0	0	0	0
1							
2							
3							

- **For filling the first item in the bag:** If we follow the above mentioned procedure, the table will look like the following.

weight→ item↓/	0	1	2	3	4	5	6
0	0	0	0	0	0	0	0
1	0	10	10	10	10	10	10
2							
3							

- **For filling the second item:**

When $jthWeight = 2$, then maximum possible profit is $\max(10, DP[1][2-2] + 15) = \max(10, 15) = 15$.

When $jthWeight = 3$, then maximum possible profit is $\max(2 \text{ not put, } 2 \text{ is put into bag}) = \max(DP[1][3], 15 + DP[1][3-2]) = \max(10, 25) = 25$.



weight→ item↓/	0	1	2	3	4	5	6
0	0	0	0	0	0	0	0
1	0	10	10	10	10	10	10
2	0	10	15	25	25	25	25
3							

- **For filling the third item:**

When $jthWeight = 3$, the maximum possible profit is $\max(DP[2][3], 40 + DP[2][3-3]) = \max(25, 40) = 40$.

When $jthWeight = 4$, the maximum possible profit is $\max(DP[2][4], 40 + DP[2][4-3]) = \max(25, 50) = 50$.

When $jthWeight = 5$, the maximum possible profit is $\max(DP[2][5], 40 + DP[2][5-3]) = \max(25, 55) = 55$.

When $jthWeight = 6$, the maximum possible profit is $\max(DP[2][6], 40 + DP[2][6-3]) = \max(25, 65) = 65$.

weight→ item↓/	0	1	2	3	4	5	6
0	0	0	0	0	0	0	0
1	0	10	10	10	10	10	10
2	0	10	15	25	25	25	25
3	0	10	15	40	50	55	65

Longest Common Subsequence (LCS)

Given two strings, **S1** and **S2**, the task is to find the length of the Longest Common Subsequence, i.e. longest subsequence present in both of the strings.

A longest common subsequence (LCS) is defined as the longest subsequence which is common in all given input sequences.

Examples:

Input: S1="ABC", S2="ACD"

Output: 2

Explanation: The longest subsequence which is present in both strings is "AC".

Input: S1 = "AGGTAB", S2 = "GXTXAYB"

Output: 4

Explanation: The longest common subsequence is "GTAB".

Longest Common Subsequence (LCS) using Bottom-Up (Tabulation):

We can use the following steps to implement the dynamic programming approach for LCS.

- Create a 2D array **dp[][]** with rows and columns equal to the length of each input string plus 1 [the number of rows indicates the indices of **S1** and the columns indicate the indices of **S2**].
- Initialize the first row and column of the dp array to 0.
- Iterate through the rows of the dp array, starting from 1 (say using iterator **i**).
 - For each **i**, iterate all the columns from **j = 1 to n**:

- If **S1[i-1]** is equal to **S2[j-1]**, set the current element of the dp array to the value of the element to (**dp[i-1][j-1] + 1**).
- Else, set the current element of the dp array to the maximum value of **dp[i-1][j]** and **dp[i][j-1]**.
- After the nested loops, the last element of the dp array will contain the length of the LCS.

See the below illustration for a better understanding:

Illustration:

Say the strings are **S1 = “AGGTAB”** and **S2 = “GXTXAYB”**.

First step: Initially create a 2D matrix (say **dp[][]**) of size 8 x 7 whose first row and first column are filled with 0.

		G	X	T	X	A	Y	B	
A G G T A B		0	1	2	3	4	5	6	7
	0	0	0	0	0	0	0	0	0
	1	0							
	2	0							
	3	0							
	4	0							
	5	0							
	6	0							

Creating dp table and filling 0 in 0th row and column

Creating

the dp table

Second step: Traverse for $i = 1$. When j becomes 5, $S1[0]$ and $S2[4]$ are equal. So the $dp[i][j]$ is updated. For the other elements take the maximum of $dp[i-1][j]$ and $dp[i][j-1]$. (In this case, if both values are equal, we have used arrows to the previous rows).

		G	X	T	X	A	Y	B
	0	1	2	3	4	5	6	7
	0	0	0	0	0	0	0	0
A	1	0	0	0	0	1	1	1
G	2	0						
G	3	0						
T	4	0						
A	5	0						
B	6	0						

Updating dp table for row 1

Filling

the row no 1

Third step: While traversed for $i = 2$, $S1[1]$ and $S2[0]$ are the same (both are 'G'). So the dp value in that cell is updated. Rest of the elements are updated as per the conditions.

		G	X	T	X	A	Y	B
	0	1	2	3	4	5	6	7
	0	0	0	0	0	0	0	0
A	1	0	0	0	0	1	1	1
G	2	0	1	1	1	1	1	1
G	3	0						
T	4	0						
A	5	0						
B	6	0						

Updating dp table for row 2

Filling

the row no. 2

Fourth step: For $i = 3$, $S1[2]$ and $S2[0]$ are again same. The updates are as follows.

		G	X	T	X	A	Y	B	
A G G T A B		0	1	2	3	4	5	6	7
	0	0	0	0	0	0	0	0	0
	1	0	0	0	0	0	1	1	1
	2	0	1	1	1	1	1	1	1
	3	0	1	1	1	1	1	1	1
	4	0							
	5	0							
6	0								

Updating dp table for row 3

Filling

row no. 3

Fifth step: For $i = 4$, we can see that $S1[3]$ and $S2[2]$ are same. So $dp[4][3]$ updated as $dp[3][2] + 1 = 2$.

		G	X	T	X	A	Y	B	
A G G T A B		0	1	2	3	4	5	6	7
	0	0	0	0	0	0	0	0	0
	1	0	0	0	0	0	1	1	1
	2	0	1	1	1	1	1	1	1
	3	0	1	1	1	1	1	1	1
	4	0	1	1	2	2	2	2	2
	5	0							
6	0								

Updating dp table for row 4

Filling row 4

Sixth step: Here we can see that for $i = 5$ and $j = 5$ the values of $S1[4]$ and $S2[4]$ are same (i.e., both are 'A'). So $dp[5][5]$ is updated accordingly and becomes 3.

		G	X	T	X	A	Y	B
	0	1	2	3	4	5	6	7
A	0	0	0	0	0	0	0	0
G	1	0	0	0	0	0	1	1
G	2	0	1	1	1	1	1	1
G	3	0	1	1	1	1	1	1
T	4	0	1	1	2	2	2	2
A	5	0	1	1	2	2	3	3
B	6	0						

Updating dp table for row 5

Filling

row 5

Final step: For $i = 6$, see the last characters of both strings are same (they are 'B'). Therefore the value of $dp[6][7]$ becomes 4.

		G	X	T	X	A	Y	B
	0	1	2	3	4	5	6	7
A	0	0	0	0	0	0	0	0
G	1	0	0	0	0	0	1	1
G	2	0	1	1	1	1	1	1
G	3	0	1	1	1	1	1	1
T	4	0	1	1	2	2	2	2
A	5	0	1	1	2	2	3	3
B	6	0	1	1	2	2	3	4

Updating dp table for row 6

Filling the final row

So we get the maximum length of common subsequence as 4.

Pseudocode:

FUNCTION longestCommonSubsequence(text1, text2):

 n = length of text1

 m = length of text2

 INITIALIZE prev and cur as vectors of size (m+1) with all elements set to 0

 FOR idx2 = 0 to m:

 cur[idx2] = 0

 FOR idx1 = 1 to n:

 FOR idx2 = 1 to m:

 IF text1[idx1-1] == text2[idx2-1]:

 cur[idx2] = 1 + prev[idx2-1]

 ELSE:

 cur[idx2] = max(cur[idx2-1], prev[idx2])

 prev = cur

 RETURN cur[m]

Time Complexity: $O(m * n)$, which remains the same.

Auxiliary Space: $O(m)$ because the algorithm uses two arrays of size m.

